

React

React là **thư viện JavaScript** giúp **xây dựng giao diện website** (UI) bằng cách **chia nhỏ từng phần** của trang web thành các **“mảnh ghép” (component)**.

Thay vì viết một file HTML, React cho phép tạo từng “thành phần” nhỏ – ví dụ như:

- Nút bấm (Button)
- Thanh menu
- Danh sách sản phẩm
- Ô nhập liệu, form đăng ký...

→ Ghép lại thành trang web hoàn chỉnh.

Điểm đặc biệt là khi dữ liệu thay đổi (ví dụ người dùng gõ tên mới), React **tự cập nhật phần giao diện liên quan**, không cần tải lại trang.

1. Môi trường

React viết bằng **JavaScript**, chạy trên **trình duyệt** (frontend).

Tuy nhiên cũng có thể:

- Dùng **React Native** → xây dựng ứng dụng **mobile** (Android/iOS)
- Dùng **Next.js** → xây dựng **website SEO tốt**, tải nhanh, chạy cả phía server
- Dùng **Electron** → xây dựng **app desktop**

=> Một nền tảng React là mở rộng được ra nhiều hướng khác nhau.

2. So sánh với các Framework

Framework	Môi trường	Khả năng học	Áp dụng
React	Thư viện JavaScript, chỉ tập trung vào giao diện.	Dễ tiếp cận, cần học thêm khái niệm component, state, hook.	Khi muốn tự do thiết kế cấu trúc, làm app web hiện đại, gọi API nhiều.
Angular	Framework hoàn chỉnh (TypeScript, có router, form, service, DI...).	Khó học hơn, cú pháp dài, cần hiểu TypeScript .	Dự án lớn, công ty cần chuẩn mực, có nhiều lập trình viên phối hợp.
Vue.js	Framework “nhẹ”, dùng HTML, CSS, JS trong cùng file .vue.	Dễ học nhất; code ngắn, dễ hiểu.	Làm web nhỏ đến trung bình, ra kết quả sớm.
Svelte	Framework kiểu “mới”, biên dịch (compile) sẵn code tối ưu.	Dễ học, code ngắn, hiệu năng cao.	Làm web nhỏ, cần nhẹ, nhanh, hoặc app nhúng vào trang có sẵn.

Next.js (dựa trên React)	Framework mở rộng React, có router, SSR, SEO mạnh.	Dễ nếu đã biết React.	Làm website chuyên nghiệp, SEO tốt, tối ưu tốc độ tải trang.
---------------------------------	--	-----------------------	--

3. Cài đặt

a. Cài Node.js

React (và Vite) chạy dựa trên **Node.js**.

Node giống như “nền” để chạy lệnh và quản lý thư viện (npm).

Vào trang: <https://nodejs.org/>

Chọn bản ổn định nhất => Node.js 22.12.0 LTS

Tải file .msi (cho Windows) hoặc .pkg (cho macOS).

Mở file cài, Next → Next → Finish (giữ nguyên mặc định, nhớ tick “Add to PATH”).

Kiểm tra sau khi cài đặt:

Mở Command Prompt (Windows) hoặc Terminal (macOS/Linux), gõ:

```
node -v
```

```
npm -v
```

Nếu thấy hiển thị:

```
v22.12.0
```

```
10.x.x
```

→ thành công

Nếu máy đã có Node rồi (và version ≥ 20.19), thì bỏ qua bước này.

b. Cài VS Code

Vào trang: <https://code.visualstudio.com/>

Tải bản dành cho Windows (User Installer).

Cài đặt bình thường (Next → Finish).

Sau khi cài xong, mở VS Code → vào File → Open Folder để mở thư mục dự án.

VS Code không bắt buộc, nhưng là công cụ phổ biến nhất để học React.

c. Sử dụng git để quản lý mã nguồn, sinh viên tìm hiểu thêm.

d. Dùng Vite để quản lý project (tạo, build)

Trước đây (khoảng 2016–2020), khi học React, mọi người thường dùng:

```
npx create-react-app my-app
```

Công cụ đó gọi là **CRA (Create React App)**.

Nó tạo ra một dự án React đầy đủ, nhưng:

Khởi động chậm: lần đầu chạy tốn vài phút mới hiện trang web.

Reload chậm: mỗi lần sửa code phải build lại, đợi lâu.

Nặng: chứa hàng trăm thư viện build (Webpack, Babel...).

Khó tùy chỉnh: muốn thêm Tailwind, alias, proxy... phải sửa nhiều file cấu hình.

Vite (đọc là “vit”, tiếng Pháp nghĩa là *nhANH*)

→ Là **công cụ khởi tạo và chạy dự án web nhanh**.

e. Tạo Project React

Tạo thư mục *demo*.

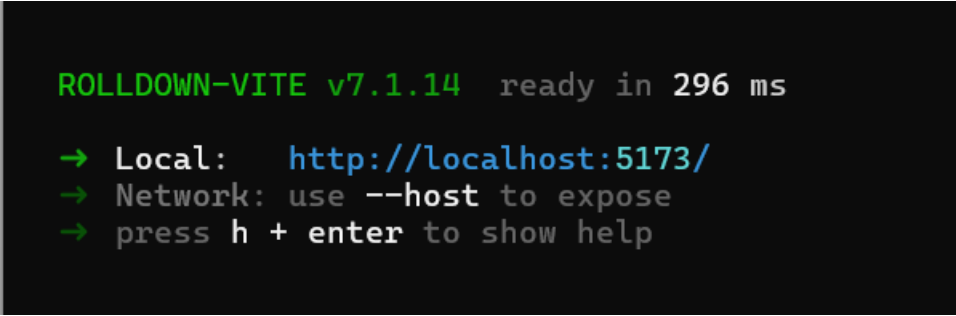
Chạy lệnh:

```
npm create vite@latest demo -- --template react
```

demo: là tên thư mục dự án, có thể đổi tùy thích.

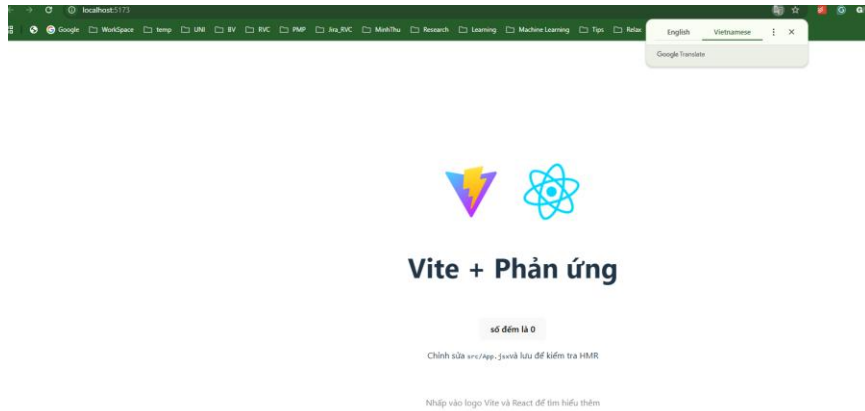
--template react: yêu cầu Vite tạo dự án mẫu React sẵn.

Sau khi cài xong sẽ thấy nội dung sau:

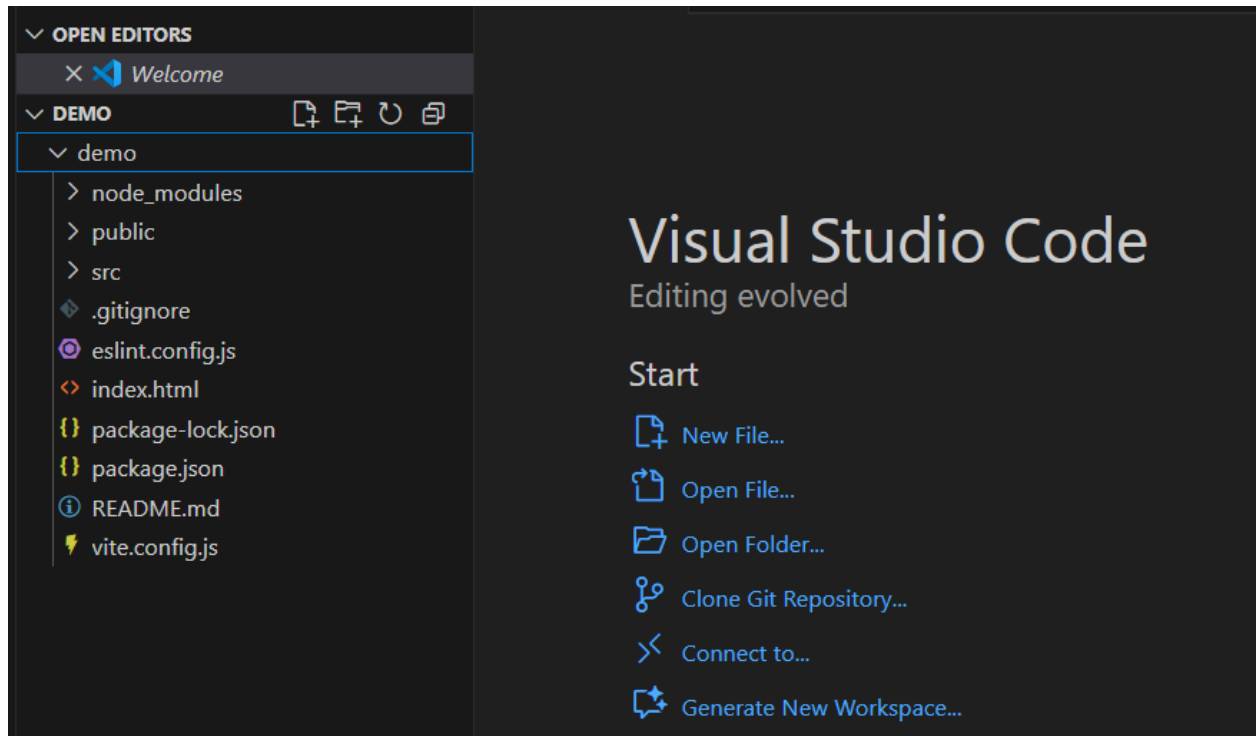


```
ROLLDOWN-VITE v7.1.14 ready in 296 ms
→ Local:    http://localhost:5173/
→ Network:  use --host to expose
→ press h + enter to show help
```

Gõ địa chỉ <http://localhost:5173> để xem thông tin.

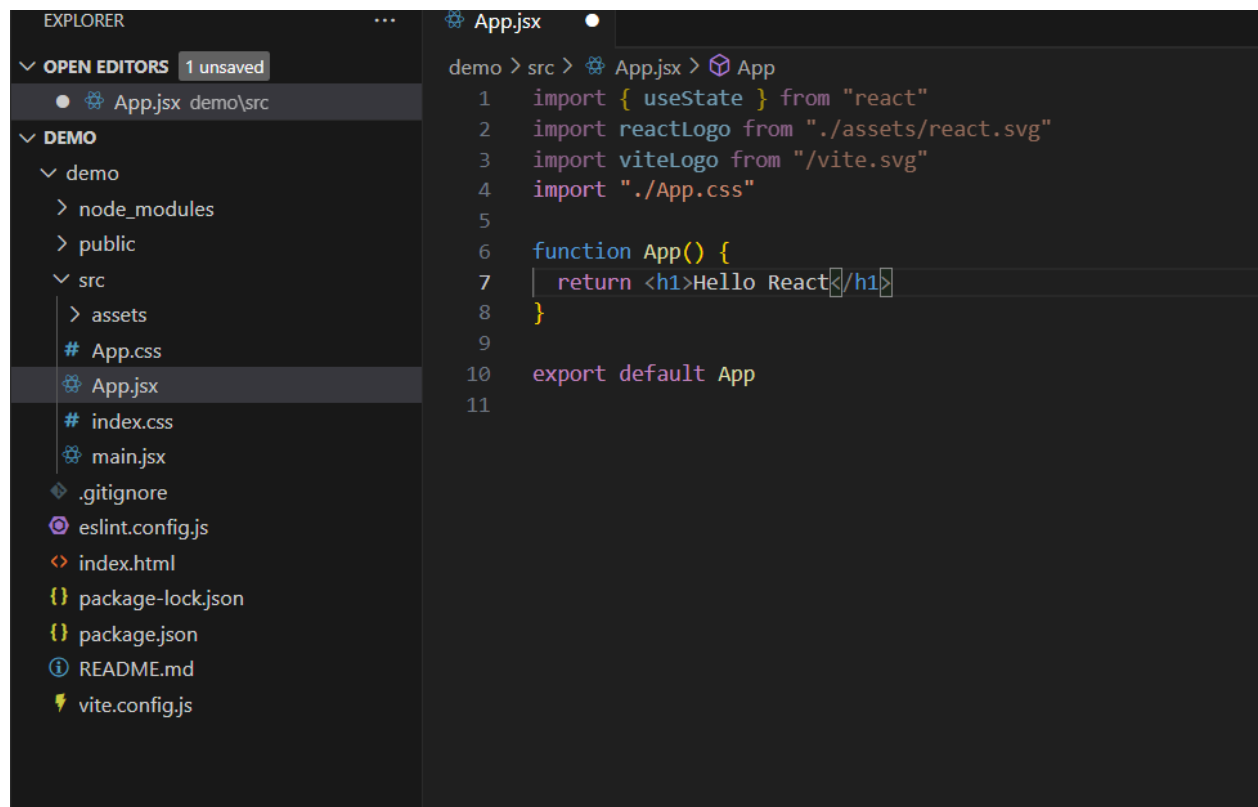


- f. Cập nhật nội dung dùng IDE: VS Code
Mở thư mục demo bằng VS Code.



Tìm file src/App.jsx → sửa lại nội dung như sau:

```
function App() {  
  return (  
    <h1>Hello React</h1>  
  );  
}  
  
export default App;
```



Save và xem kết quả nội dung trình duyệt tự động cập nhật.

4. Các thành phần cơ bản

- React không phải HTML, mà là JavaScript tạo ra HTML.
- Mọi thứ trong React đều là component (thành phần giao diện).
- Mỗi component là một hàm (function) trả về đoạn giao diện được viết bằng JSX.

JSX = JavaScript XML

→ Là cách viết HTML nằm trong JavaScript, giúp viết UI trực quan.

Ví dụ:

```
function Hello() {  
  return <h1>Hello React </h1>;  
}
```

Dòng <h1>...</h1> là JSX.

React sẽ dịch JSX thành lệnh JavaScript tạo ra thẻ HTML tương ứng trên trang web.

Lưu ý:

JSX chỉ được phép có 1 thẻ parent bọc ngoài.

Dùng className thay vì class.

Có thể chèn biến bằng {}.

Ví dụ:

```
const name = "Minh";  
return <h2>Xin chào, {name}!</h2>;
```

Component =>

- Là “mảnh giao diện độc lập” có thể tái sử dụng.
- Có thể coi component như “hàm trả về một khối UI”.

Ví dụ: tạo file src/components/Hello.jsx

```
export default function Hello() {  
  return <h2>Xin chào sinh viên!</h2>;  
}
```

Dùng component này trong App.jsx:

```
import Hello from "../components/Hello";

export default function App() {
  return (
    <div>
      <h1>Ứng dụng React đầu tiên</h1>
      <Hello />
    </div>
  );
}
```

Props =>

- Props là “tham số truyền vào” component (giống như truyền biến vào hàm).
- Dùng để tái sử dụng component với nhiều giá trị khác nhau.

Ví dụ:

```
function Welcome(props) {
  return <h2>Xin chào, {props.name}!</h2>;
}

export default function App() {
  return (
    <>
      <Welcome name="An" />
      <Welcome name="Bình" />
      <Welcome name="Chi" />
    </>
  );
}
```

State =>

- Là nơi lưu dữ liệu có thể thay đổi (ví dụ: số lần bấm nút).

- Khi **state thay đổi**, React **tự động vẽ lại giao diện**.

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Bạn đã bấm {count} lần</p>
      <button onClick={() => setCount(count + 1)}>Click here!</button>
    </div>
  );
}
```

Giải thích:

- `useState(0)` tạo ra biến `count` (giá trị hiện tại) và `setCount`(đổi giá trị).
- Mỗi khi gọi `setCount`, React **tự động cập nhật giao diện** .

useEffect =>

Khi muốn **chạy một đoạn code phụ** (gọi API, in ra console, hay thay đổi tiêu đề trang), React dùng **useEffect**.

```
import { useEffect, useState } from "react";

export default function UserList() {
  const [users, setUsers] = useState([]);

  // Gọi API khi component được hiển thị
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []); // [] để chỉ chạy 1 lần khi trang load

  return (
    <div>
      <h2>Danh sách người dùng</h2>
      <ul>
```



```

        {users.map(u => <li key={u.id}>{u.name}</li>)}
      </ul>
    </div>
  );
}

```

Lệnh	Ý nghĩa
<code>const [users, setUsers] = useState([])</code>	Tạo state lưu danh sách người dùng. Ban đầu là mảng rỗng.
<code>useEffect(() => {...}, [])</code>	Khi component xuất hiện lần đầu , chạy đoạn code bên trong.
<code>fetch(...)</code>	Gọi API (giống gửi request lên server).
<code>.then(res => res.json())</code>	Chuyển dữ liệu nhận được sang dạng JSON.
<code>.then(data => setUsers(data))</code>	Gán dữ liệu vào users → React tự render lại giao diện.
<code>{users.map(...)}</code>	Duyệt qua mảng người dùng, hiển thị từng tên lên màn hình.

React Router =>

- Trong web bình thường, mỗi khi chuyển trang là trình duyệt tải lại trang HTML khác.
- Với React, không muốn “load lại toàn bộ”, mà chỉ thay đổi phần nội dung chính => React Router giúp làm điều đó.

Lưu ý: cài react-router-dom: *npm install react-router-dom*

```

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Home() {
  return <h2>Trang chủ</h2>;
}

function About() {
  return <h2>Giới thiệu</h2>;
}

```

```

export default function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Trang chủ</Link> | <Link to="/about">Giới thiệu</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

```

Lệnh	Ý nghĩa
import {...} from "react-router-dom"	Import các thành phần cần thiết để định tuyến.
<BrowserRouter>	Bao quanh toàn bộ ứng dụng – giống “bản đồ định hướng”.
<Link to="/">Trang chủ</Link>	Thẻ điều hướng nội bộ trong React (không reload trang).
<Routes> & <Route>	Định nghĩa đường dẫn và component tương ứng.
element={<Home />}	Khi URL là "/", hiển thị component Home.
element={<About />}	Khi URL là "/about", hiển thị component About.

Form =>

- Trong React, form được “điều khiển” bởi **state**.
- Nghĩa là mỗi ô input sẽ **liên kết với một biến trong component**.
- Khi người dùng gõ, cập nhật lại state → React tự hiển thị lại.

```
import { useState } from "react";
```

```

export default function ContactForm() {
  const [name, setName] = useState("");
  const [msg, setMsg] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // Không reload trang
    alert(`Tên: ${name}\nTin nhắn: ${msg}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>Tên:</label>
      <input value={name} onChange={e => setName(e.target.value)} />

      <label>Tin nhắn:</label>
      <textarea value={msg} onChange={e => setMsg(e.target.value)} />

      <button type="submit">Gửi</button>
    </form>
  );
}

```

Lệnh	Ý nghĩa
useState("")	Biến lưu nội dung của input.
value={name}	Input hiển thị đúng giá trị hiện tại trong state.
onChange={e => setName(e.target.value)}	Khi người dùng gõ, cập nhật lại state.
onSubmit={handleSubmit}	Khi form bấm “Gửi”, chạy hàm handleSubmit.
e.preventDefault()	Chặn hành vi mặc định (không reload trang).
alert(...)	Hiển thị kết quả vừa nhập.

Sinh viên cần tìm thêm các **từ khóa** bên dưới:

JSX, Component, Form, Props, State, Event Handling, Conditional Rendering, List & key, useState, useEffect, useRef

Nguồn tham khảo:

<https://react.dev/learn>

YouTube: *Net Ninja – React Beginner Tutorial*

Một số hiệu ứng giúp trang web lôi cuốn hơn:

Chủ đề	Nguồn
Framer Motion cơ bản	https://www.framer.com/motion/
Toast notification	https://react-hot-toast.com/docs
Tailwind transition	https://tailwindcss.com/docs/transition-property
Dark Mode	https://tailwindcss.com/docs/dark-mode
Skeleton & hover effect	https://flowbite-react.com/docs/components/skeleton/
Animate on Scroll (AOS)	https://michalsnik.github.io/aos/

Sinh viên tham khảo để ứng dụng vào bài tập.

Bài tập:

Hãy xây dựng một web app đặt món ăn đơn giản cho quán “**ReactFood**”. Khách hàng có thể xem menu món ăn, tìm kiếm, xem chi tiết, thêm vào giỏ hàng, và gửi đơn đặt hàng qua một form.

Yêu cầu chi tiết:

Màn hình	Nội dung chính
Trang chủ (/)	Giới thiệu “ReactFood”, nút “Xem Menu”
Menu (/menu)	Danh sách món ăn (ảnh, tên, giá), search + filter category
Chi tiết món (/menu/:id)	Hiển thị chi tiết món, mô tả, nút “Thêm vào giỏ”
Giỏ hàng (/cart)	Danh sách món đã chọn, tăng/giảm số lượng, xóa món (Modal confirm), tính tổng tiền
Đặt hàng (/checkout)	Form nhập tên, địa chỉ, điện thoại, xác nhận đơn (mock API)
(Tùy chọn) Đăng nhập giả (/login)	Nhập email/password, lưu vào localStorage.

Lưu ý: Đây là trang web tĩnh, chưa cần tích API.

Hãy hoàn chỉnh web app trên dùng reactjs, quay video và giải thích source code mà mình đã làm, video upload lên youtube, gửi link.

Kịch bản quay video như sau:

1. Giới thiệu thông tin sinh viên (em tên..., msssv.....)
2. Giới thiệu mỗi chức năng gồm: chức năng làm Page gì? Mở source code giải thích dùng component như thế nào? Công nghệ hoặc kiến thức áp dụng như thế nào?

Video tối đa 10 phút.